to be compatible with the kernel into which it is going to be loaded. The *MODULE_** macros populate module-related information, which acts like the module's 'signature'.

Building our first Linux driver

Once we have the C code, it is time to compile it and create the module file ofd.ko. We use the kernel build system to do this. The following *Makefile* invokes the kernel's build system from the kernel source, and the kernel's Makefile will, in turn, invoke our first driver's Makefile to build our first driver. To build a Linux driver, you need to have the kernel source (or atleast the kernel headers) installed on your system. The kernel source is assumed to be installed at */usr/src/linux*. If it's at any other location on your system, specify the location in the KERNEL_SOURCE variable in this *Makefile*.

```
# Makefile – makefile of our first driver

# if KERNELRELEASE is defined, we've been
invoked from the
# kernel build system and can use its
language.
ifneq (${KERNELRELEASE},)
    obj-m := ofd.o
# Otherwise we were called directly from
the command line.
# Invoke the kernel build system.
else
    KERNEL_SOURCE := /usr/src/linux
    PWD := $(shell pwd)
default:
    ${MAKE} -C ${KERNEL_SOURCE}
SUBDIRS=${PWD} modules

clean:
    ${MAKE} -C ${KERNEL_SOURCE}
SUBDIRS=${PWD} clean
endif
```

With the C code (*ofd.c*) and *Makefile* ready, all we need to do is invoke *make* to build our first driver (*ofd.ko*).

```
$ make
make -C /usr/src/linux SUBDIRS=... modules
make[1]: Entering directory `/usr/src/
linux'
  CC [M]  .../ofd.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC      .../ofd.mod.o
  LD [M]  .../ofd.ko
make[1]: Leaving directory `/usr/src/
linux'
```

Summing up

Once we have the *ofd.ko* file, perform the usual steps as the root user, or with *sudo*.

```
# su
# insmod ofd.ko
# lsmod | head -10
```

lsmod should show you the *ofd* driver loaded.

While the students were trying their first module, the bell rang, marking the end of the session. Professor Gopi concluded, "Currently, you may not be able to observe anything other than the "*lsmod*" listing showing the driver has loaded. Where's the *printk* output gone? Find that out for yourselves, in the lab session, and update me with your findings. Also note that our first driver is a template for any driver you would write in Linux. Writing a specialised driver is just a matter of what gets filled into its constructor and destructor. So, our further learning will be to enhance this driver to achieve specific driver functionalities." END

By: Anil Kumar Pugalia

The author is a freelance trainer in Linux internals, Linux device drivers, embedded Linux and related topics. Prior to this, he had worked at Intel and Nvidia. He has been exploring Linux since 1994. A gold medallist from the Indian Institute of Science, Linux and knowledge-sharing are two of his many passions. More information about him can be found at http://profession.sarika-pugs.com. He can be reached at email@sarika-pugs.com